МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №6

«Автоматизоване тестування Python-програм»

Варіант №5 (Каталог автомобілів)

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
проф. Свелеба Сергій Андрійович

Львів — 2026

Хід роботи

Тема: Автоматизоване тестування Python-програм

Мета: Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку `pytest`, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

Варіант: 5 (Каталог автомобілів)

---

1. Короткий опис тестованого проєкту

Проєкт являє собою модульну систему обліку та аналітики каталогу автомобілів `car-catalog`. Архітектура побудована на засадах Domain-Driven Design (DDD) та SOLID: - **Domain Entities & Value Objects:** Сутності `Vehicle`, `Car`, `ElectricCar` із бізнес-правилами та валідацією полів, Value Objects `Money`, `Mileage`, `VIN`. - **Сервісний шар (Services):** `CatalogService` для операцій додавання, пошуку, фільтрації, чекауту та розрахунку статистики; `VehicleEnrichmentService` для інтеграції із зовнішніми джерелами специфікацій; `AsyncVehicleMarketService` для асинхронної ринкової оцінки. - **Потокова обробка та валідація:** `parse_car_record` із строгою валідацією типів, діапазонів значень та генерацією `RecordValidationError` із збереженням першопричини (`Exception Chaining`); `import_cars` у strict та tolerant режимах. - **Атомарний експорт:** `AtomicFileWriter` та експортери `JsonExporter`, `JsonlExporter`, `CsvExporter`, `YamlExporter`. - **Конфігурація:** `AppConfig`, завантаження та fail-fast валідація діапазонів років і цін із файлів YAML та перевизначення через змінні оточення.

---

2. Структура Production Code

```
src/car_catalog/
├── __init__.py           # Версія пакета (0.6.0)
├── py.typed              # Маркер PEP 561 суворої типізації
├── abstractions.py       # Абстрактні базові класи та DiscountPolicy
├── analytics.py          # Функції аналітичних розрахунків
├── batches.py            # Пакетна обробка генераторів
├── comparisons.py        # Порівняння та сортування
├── config.py             # AppConfig та валідація YAML
├── criteria.py           # Патерн Specification (SearchCriteria)
├── data.py               # Демонстраційні датасети
├── decorators.py         # Декоратори вимірювання часу
├── dto.py                # TypedDict та DTO результатів
├── exceptions.py         # Ієрархія ApplicationError
├── exporters.py          # Експортери JSON, JSONL, CSV, YAML
├── filters.py            # Потокові фільтри
├── iterators.py          # Користувацькі ітератори
├── models.py             # Доменні моделі Vehicle, Car, ElectricCar, VehicleCatalog
├── notifiers.py          # Повідомлення VehicleNotifier
├── parsers.py            # Парсери рядків
├── pipeline.py           # Конвеєри обробки
├── processors.py         # Агрегації, індекси, Counter, defaultdict
├── protocols.py          # Typing Protocol для зовнішніх інтерфейсів
├── readers.py            # Потокове зчитування CSV/JSON/YAML
├── repositories.py       # InMemoryRepository, VehicleRepository
├── services.py           # Фасадні сервіси та асинхронні конвеєри
├── specifications.py     # Специфікація характеристик авто
├── streaming_analytics.py# Потокова аналітика великих даних
├── transformations.py    # Трансформації цін та валют
├── utils.py              # atomic_file_writer, logged_operation, timer
├── validators.py         # parse_car_record та валідатори
└── value_objects.py      # Money, Mileage, VIN
```

3. Структура Test Suite

Тестовий набір організовано згідно з рекомендаціями стандартів `pytest` та методичних вказівок (Розділи 8 та 102):

```
tests/
├── conftest.py                  # Глобальні фікстури, фабрики, yield-тести, autouse
├── unit/                        # Модульні ізольовані тести (швидкі, детерміновані)
│   ├── test_models.py           # Валідація полів, граничні значення, dunders, Value Objects
│   ├── test_services.py         # Сервіси, FakeRepository, Mocks, side_effect, AsyncMock
│   ├── test_validators.py       # Параметризовані тести записів, exception chaining
│   ├── test_configuration.py    # YAML, граничні діапазони років/цін, monkeypatch
│   └── test_exporters.py        # Атомарний запис, ізоляція tmp_path, формати експорту
├── integration/                 # Інтеграційні тести компонентів
│   ├── test_import_pipeline.py  # Потоковий імпорт у tolerant та strict режимах
│   ├── test_export_pipeline.py  # Імпорт -> фільтрація -> атомарний експорт
│   └── test_application_flow.py # Повний наскрізний цикл та асинхронний пайплайн
├── test_car_catalog.py          # Тести зворотної сумісності (Лаб №1-3)
├── test_oop_lab4.py             # Тести ООП моделі (Лаб №4)
└── test_reliable_lab5.py        # Тести надійності та винятків (Лаб №5)
```

4. Test Matrix (Матриця тестування)

Категорія сценарію	Тестований компонент	Вхідні дані	Очікувана поведінка
Normal Case	`Car.__init__`	id=1, Toyota, Camry, 2021, 26000.0, 45000.0	Створення об'єкта, доступ до властивостей
Minimum Boundary	`Car.year`	year=1886	Валідний об'єкт (рік створення першого авто)
Maximum Boundary	`Car.year`	year=2026	Валідний об'єкт (поточний виробничий рік)
Invalid Boundary	`Car.year`	year=1885, year=0, year=-2020	Генерація `ValueError("Invalid manufacturing year")`
Minimum Boundary	`Car.price`	price=0.0, price=0.01	Дозволено (безкоштовне або мінімальне авто)
Invalid Boundary	`Car.price`	price=-0.01	Генерація `ValueError("Price cannot be negative")`
Invalid Boundary	`Car.mileage`	mileage=-1.0	Генерація `ValueError("Mileage cannot be negative")`
Empty Input	`Car.brand`, `model`	brand="", model=""	Генерація `ValueError("Brand and model cannot be empty")`
Empty Collection	`VehicleCatalog`	vehicles=[]	`most_expensive() is None`, `average_price() == 0.0`
Invalid Record	`parse_car_record`	"brand": "", "year": "bad"	`RecordValidationError` із прив'язкою до рядка
Exception Chaining	`parse_car_record`	"year": "invalid_year"	`exc_info.value.__cause__ is ValueError`
Missing File	`load_config`	"does_not_exist.yaml"	`ConfigurationError("Configuration file not found")`
Config Range Conflict	`validate_config`	min_year=2025, max_year=2010	`ConfigurationError("max_year cannot be less than min_year")`
Dependency Success	`VehicleEnrichmentService`	mock `return_value`	Повернення збагачених специфікацій авто
Dependency Failure	`VehicleEnrichmentService`	mock `side_effect=ConnectionError`	Прокидання винятку, зупинка ланцюжка
Unwanted Call	Failure path	mock notifier	Перевірка `notifier.send.assert_not_called()`
Async External Call	`AsyncVehicleMarketService`	`AsyncMock`	`assert_awaited_once_with`, повернення оцінки
Atomic File Rollback	`atomic_file_writer`	збій під час запису	Збереження цільового файлу, очищення `.tmp`

5. Приклади Unit Tests

Перевірка доменної моделі та обчислень

```
@pytest.mark.unit
def test_car_creation_and_attributes(sample_car: Car) -> None:
    assert sample_car.id == 1
    assert sample_car.brand == "Toyota"
    assert sample_car.model == "Camry"
    assert sample_car.year == 2021
    assert sample_car.price == pytest.approx(26000.0)
    assert sample_car.mileage == pytest.approx(45000.0)
    assert sample_car.full_name == "Toyota Camry"
    assert sample_car.age == 2026 - 2021
```

Перевірка колекції `VehicleCatalog`

```
@pytest.mark.unit
def test_vehicle_catalog_collection(sample_cars_list: list[Vehicle], sample_car: Car) -> None:
    catalog = VehicleCatalog(sample_cars_list)
    assert len(catalog) == 5
    assert catalog[0].id == 1
    assert 1 in catalog
    assert 999 not in catalog

    most_exp = catalog.most_expensive()
    assert most_exp is not None
    assert most_exp.brand == "BMW"

    avg_p = catalog.average_price()
    assert avg_p > 0.0
```

---

6. Приклад Integration Test

Тест перевіряє інтеграцію читача CSV, фільтра за роком та ціною, і експортера в JSON:

```
@pytest.mark.integration
def test_filter_and_export_pipeline_end_to_end(sample_csv_file: Path, tmp_path: Path) -> None:
    output_file = tmp_path / "filtered_cars.json"
    stats = ImportStatistics()
    cars_stream = import_cars(sample_csv_file, skip_invalid=False, statistics=stats)

    filter_criteria = FilterConfig(
        min_year=2022,
        max_year=2024,
        min_price=30000.0,
        max_price=80000.0,
    )

    filtered_cars = list(filter_imported_cars(cars_stream, filter_criteria))
    assert len(filtered_cars) == 2

    exporter = JsonExporter()
    exported_count = exporter.export(filtered_cars, output_file)

    assert exported_count == 2
    assert output_file.exists()

    data = json.loads(output_file.read_text(encoding="utf-8"))
    assert len(data) == 2
    models = {car_dict["model"] for car_dict in data}
    assert models == {"M3", "Model 3"}
```

---

7. Приклади Fixtures та Fixture Factory

Shared Fixture у `conftest.py`

```
@pytest.fixture
def sample_car() -> Car:
    return Car(
        id=1,
        brand="Toyota",
        model="Camry",
        year=2021,
        price=26000.0,
        mileage=45000.0,
    )
```

Fixture Factory (Розширені вимоги, Розділ 96)

```
@pytest.fixture
def car_factory() -> Callable[..., Car]:
    def _create_car(
        *,
        car_id: int = 10,
        brand: str = "Toyota",
        model: str = "Corolla",
        year: int = 2020,
        price: float = 20000.0,
        mileage: float = 30000.0,
    ) -> Car:
        return Car(
            id=car_id,
            brand=brand,
            model=model,
            year=year,
            price=price,
            mileage=mileage,
        )
    return _create_car
```

Fixture із Teardown (`yield`)

```
@pytest.fixture
def teardown_tracker() -> Generator[dict[str, Any], None, None]:
    tracker: dict[str, Any] = {"initialized": True, "cleaned_up": False}
    yield tracker
    tracker["cleaned_up"] = True
```

Autouse Fixture

```
@pytest.fixture(autouse=True)
def clean_app_env(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.delenv("APP_SKIP_INVALID", raising=False)
    monkeypatch.delenv("APP_LOG_LEVEL", raising=False)
```

---

8. Приклади Parametrization та Boundary Tests

Параметризація з явними ідентифікаторами `ids` через `pytest.param`:

```
@pytest.mark.unit
@pytest.mark.parametrize(
    ("year", "expected"),
    [
        pytest.param(1886, 1886, id="lower_boundary_first_car"),
        pytest.param(2000, 2000, id="normal_year_2000"),
        pytest.param(2026, 2026, id="upper_boundary_current_year"),
    ],
)
def test_car_valid_boundary_years(car_factory: Callable[..., Car], year: int, expected: int) -> None:
    car = car_factory(year=year)
    assert car.year == expected

@pytest.mark.unit
@pytest.mark.parametrize(
    "invalid_year",
    [
        pytest.param(1885, id="below_minimum_1886"),
        pytest.param(0, id="zero_year"),
        pytest.param(-2020, id="negative_year"),
    ],
)
def test_car_invalid_boundary_years(car_factory: Callable[..., Car], invalid_year: int) -> None:
    with pytest.raises(ValueError, match="Invalid manufacturing year"):
        car_factory(year=invalid_year)
```

---

9. Приклади `pytest.raises`, `pytest.approx`, `tmp_path`, `monkeypatch`

`pytest.raises` та Exception Chaining

```
@pytest.mark.unit
def test_exception_chaining_on_invalid_numeric_conversion() -> None:
    record = {
        "vehicle_id": "1",
        "brand": "Toyota",
        "model": "Camry",
        "year": "invalid_numeric_year",
        "price": "26000.0",
        "mileage": "45000",
    }
    with pytest.raises(RecordValidationError) as exc_info:
        parse_car_record(record, line_number=10)

    assert exc_info.value.field == "year"
    assert exc_info.value.__cause__ is not None
    assert isinstance(exc_info.value.__cause__, ValueError)
```

`pytest.approx` для плаваючих чисел

```
avg_price = service.calculate_average_price()
expected_avg = (26000.0 + 42000.0 + 75000.0 + 18500.0 + 38000.0) / 5
assert avg_price == pytest.approx(round(expected_avg, 2))
```

`tmp_path` для герметичності файлової системи

```
@pytest.mark.unit
def test_atomic_file_writer_success(tmp_path: Path) -> None:
    target_file = tmp_path / "atomic_success.txt"
    with atomic_file_writer(target_file) as file_handle:
        file_handle.write("Atomic content payload")

    assert target_file.exists()
    assert target_file.read_text(encoding="utf-8") == "Atomic content payload"
```

`monkeypatch` для безпечної зміни конфігурації

```
@pytest.mark.unit
def test_config_env_overrides_with_monkeypatch(
    sample_yaml_config: Path,
    monkeypatch: pytest.MonkeyPatch,
) -> None:
    monkeypatch.setenv("APP_SKIP_INVALID", "false")
    monkeypatch.setenv("APP_LOG_LEVEL", "DEBUG")

    config = load_config(sample_yaml_config)
    assert config.processing.skip_invalid is False
    assert config.logging.level == "DEBUG"
```

---

10. Приклади Mock, `return_value`, `side_effect`, Interaction Assertions

Успішний виклик із перевіркою взаємодії (`assert_called_once_with`)

```
@pytest.mark.unit
def test_vehicle_enrichment_service_mock_success() -> None:
    external_mock = Mock(spec=ExternalVehicleInfoService)
    external_mock.fetch_vehicle_specs.return_value = {
        "engine": "2.5L 4-Cylinder",
        "horsepower": 203,
        "transmission": "8-Speed Automatic",
    }
    enrichment_service = VehicleEnrichmentService(external_mock)
    vin = "1HGCR2F83HA123456"

    specs = enrichment_service.enrich_specs(vin)
    assert specs["horsepower"] == 203
    external_mock.fetch_vehicle_specs.assert_called_once_with(vin)
```

Збій залежності через `side_effect`

```
@pytest.mark.unit
def test_vehicle_enrichment_service_mock_side_effect_failure() -> None:
    external_mock = Mock(spec=ExternalVehicleInfoService)
    external_mock.fetch_vehicle_specs.side_effect = ConnectionError("External service timeout")
    enrichment_service = VehicleEnrichmentService(external_mock)
    vin = "1HGCR2F83HA123456"

    with pytest.raises(ConnectionError, match="External service timeout"):
        enrichment_service.enrich_specs(vin)

    external_mock.fetch_vehicle_specs.assert_called_once_with(vin)
```

---

11. Приклад асинхронного тестування через `AsyncMock`

Для тестування асинхронного клієнта ринкової оцінки використано `AsyncMock` та маркер `@pytest.mark.asyncio`:

```
@pytest.mark.unit
@pytest.mark.asyncio
async def test_async_vehicle_market_service_success() -> None:
    client_mock = AsyncMock(spec=AsyncExternalMarketClient)
    client_mock.fetch_market_evaluation.return_value = {
        "brand": "Toyota",
        "model": "Camry",
        "year": 2021,
        "estimated_price": 25500.0,
    }
    service = AsyncVehicleMarketService(client_mock)

    price = await service.get_market_price("Toyota", "Camry", 2021)
    assert price == pytest.approx(25500.0)
    client_mock.fetch_market_evaluation.assert_awaited_once_with("Toyota", "Camry", 2021)
```

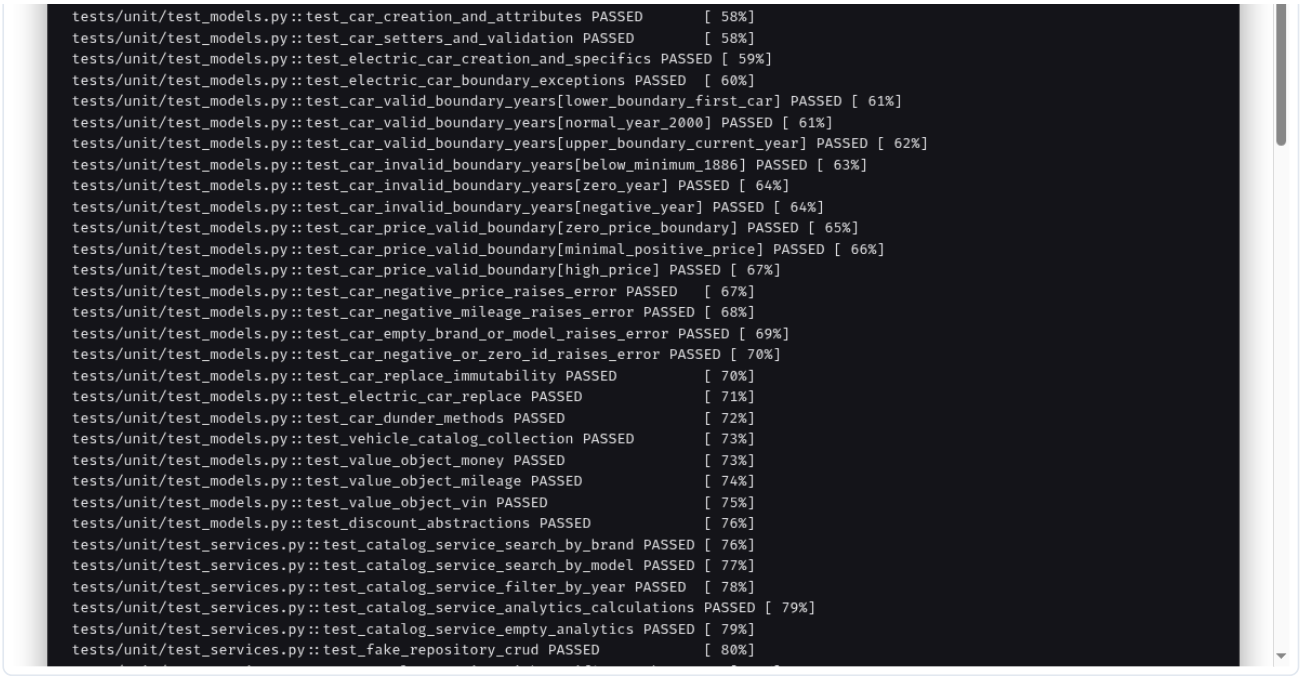
---

12. Результати запуску тестів та звіт про покриття (Coverage)

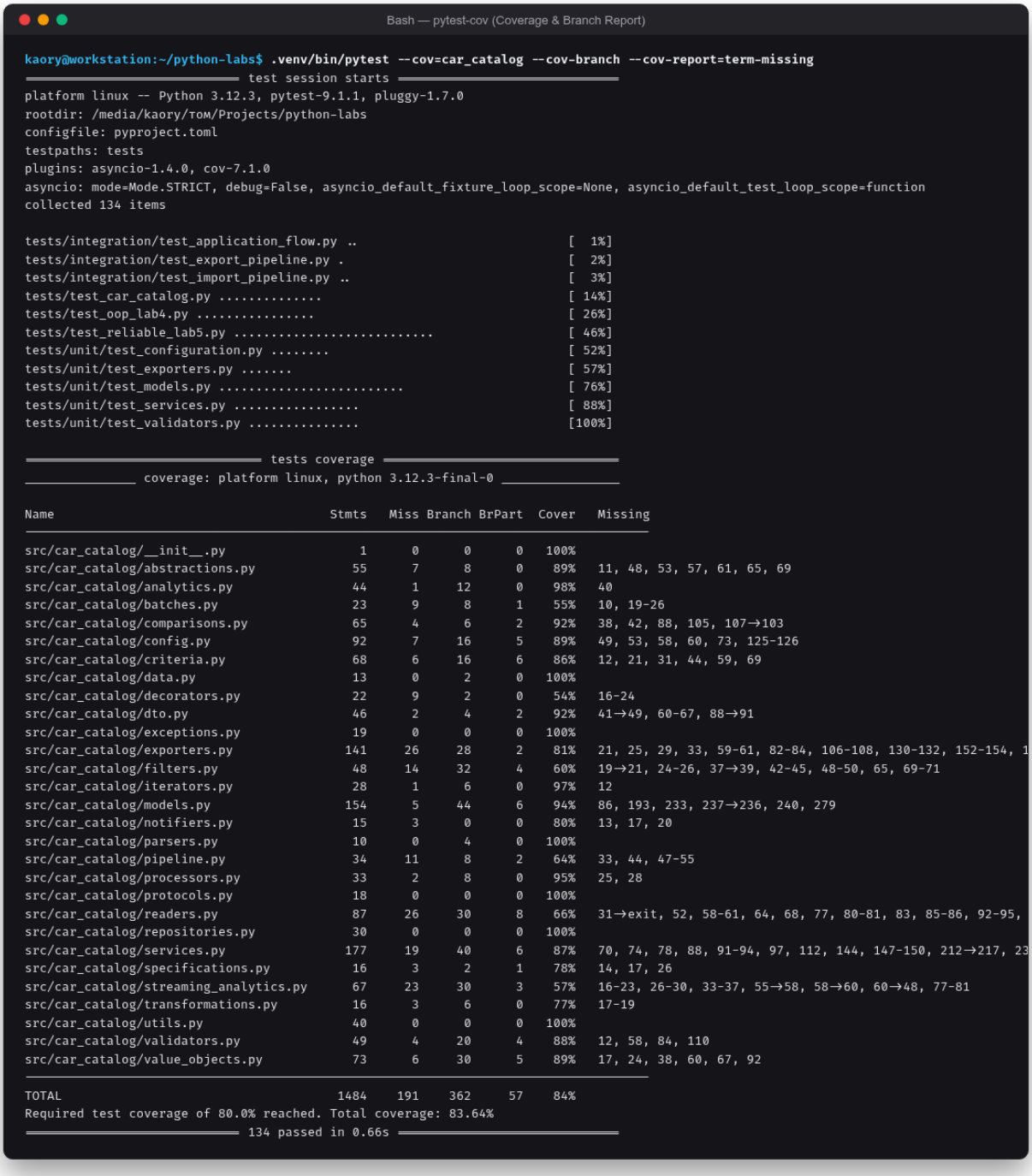
Загальна статистика проходження тестів

- **Всього зібрано та виконано тестів:** 134
- **Успішно пройдено (Passed):** 134 (100%)
- **Провалено (Failed):** 0
- **Час виконання test suite:** 0.89 с


Bash — pytest -v (Lab 6 Test Suite)
kaory@workstation:~/python-labs$ .venv/bin/pytest -v
========== test session starts ==========
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.7.0 -- /media/kaory/том/Projects/python-labs/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /media/kaory/том/Projects/python-labs
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collecting ... collected 134 items
tests/integration/test_application_flow.py::test_full_application_pipeline_execution PASSED [0%]
tests/integration/test_application_flow.py::test_async_application_pipeline_flow PASSED [1%]
tests/integration/test_export_pipeline.py::test_filter_and_export_pipeline_end_to_end PASSED [2%]
tests/integration/test_import_pipeline.py::test_streaming_import_tolerant_mode PASSED [2%]
tests/integration/test_import_pipeline.py::test_streaming_import_strict_mode_aborts PASSED [3%]
tests/test_car_catalog.py::TestStreamingReadersAndParsers::test_infinite_car_stream PASSED [4%]
tests/test_car_catalog.py::TestStreamingReadersAndParsers::test_read_lines_and_clean PASSED [5%]
tests/test_car_catalog.py::TestStreamingReadersAndParsers::test_read_multiple_files PASSED [5%]
tests/test_car_catalog.py::TestStreamingFiltersAndValidation::test_filter_by_brand_and_year_stream PASSED [6%]
tests/test_car_catalog.py::TestStreamingFiltersAndValidation::test_validate_cars PASSED [7%]
tests/test_car_catalog.py::TestTransformationsAndBatches::test_apply_price_discount PASSED [8%]
tests/test_car_catalog.py::TestTransformationsAndBatches::test_batched_cars PASSED [8%]
tests/test_car_catalog.py::TestTransformationsAndBatches::test_normalize_car_names PASSED [9%]
tests/test_car_catalog.py::TestIteratorsAndAnalytics::test_custom_iterator_and_iterable PASSED [10%]
tests/test_car_catalog.py::TestIteratorsAndAnalytics::test_pairwise_and_accumulate PASSED [11%]
tests/test_car_catalog.py::TestIteratorsAndAnalytics::test_recursive_flatten PASSED [11%]
tests/test_car_catalog.py::TestIteratorsAndAnalytics::test_streaming_analytics PASSED [12%]
tests/test_car_catalog.py::TestPipelineAndCompatibility::test_backwards_compatibility PASSED [13%]
tests/test_car_catalog.py::TestPipelineAndCompatibility::test_end_to_end_pipeline PASSED [14%]
tests/test_oop_lab4.py::TestValueObjects::test_mileage_and_vin PASSED [14%]
tests/test_oop_lab4.py::TestValueObjects::test_money_currency_mismatch PASSED [15%]
tests/test_oop_lab4.py::TestValueObjects::test_money_valid_operations PASSED [16%]
tests/test_oop_lab4.py::TestValueObjects::test_money_validation_and_immutability PASSED [17%]
tests/test_oop_lab4.py::TestDomainEntitiesAndInheritance::test_car_properties_and_encapsulation PASSED [17%]
tests/test_oop_lab4.py::TestDomainEntitiesAndInheritance::test_electric_car_inheritance_and_specifics PASSED [18%]
tests/test_oop_lab4.py::TestDomainEntitiesAndInheritance::test_replace_methods PASSED [19%]
tests/test_oop_lab4.py::TestVehicleCatalogCollection::test_analytics PASSED [20%]
tests/test_oop_lab4.py::TestVehicleCatalogCollection::test_dunder_methods PASSED [20%]
tests/test_oop_lab4.py::TestSearchCriteriaStrategy::test_composite_criteria PASSED [21%]
tests/test_oop_lab4.py::TestSearchCriteriaStrategy::test_criteria_matches PASSED [22%]
tests/test_oop_lab4.py::TestSearchCriteriaStrategy::test_runtime_checkable_protocol PASSED [23%]
tests/test_oop_lab4.py::TestGenericRepositoryAndDI::test_catalog_service_with_dependency_injection PASSED [23%]
tests/test_oop_lab4.py::TestGenericRepositoryAndDI::test_repository_operations PASSED [24%]
tests/test_oop_lab4.py::TestDTOAndComparisons::test_comparisons_and_architectures PASSED [25%]
tests/test_oop_lab4.py::TestDTOAndComparisons::test_dto_payload_conversion PASSED [26%]
tests/test_reliable_lab5.py::TestExceptionHierarchy::test_inheritance PASSED [26%]
tests/test_reliable_lab5.py::TestExceptionHierarchy::test_record_validation_error_attributes PASSED [27%]
tests/test_reliable_lab5.py::TestRecordValidation::test_empty_brand_and_model PASSED [28%]
tests/test_reliable_lab5.py::TestRecordValidation::test_invalid_vehicle_id PASSED [29%]
tests/test_reliable_lab5.py::TestRecordValidation::test_negative_price_and_mileage PASSED [29%]
tests/test_reliable_lab5.py::TestRecordValidation::test_negative_vehicle_id PASSED [30%]
tests/test_reliable_lab5.py::TestRecordValidation::test_valid_record_with_id_fallback PASSED [31%]
tests/test_reliable_lab5.py::TestRecordValidation::test_valid_record_with_vehicle_id PASSED [32%]
tests/test_reliable_lab5.py::TestRecordValidation::test_year_validation_and_chaining PASSED [32%]
tests/test_reliable_lab5.py::TestConfigManagement::test_env_override PASSED [33%]
tests/test_reliable_lab5.py::TestConfigManagement::test_invalid_yaml PASSED [34%]
tests/test_reliable_lab5.py::TestConfigManagement::test_load_and_validate_valid_config PASSED [35%]
tests/test_reliable_lab5.py::TestConfigManagement::test_missing_config_file PASSED [35%]
tests/test_reliable_lab5.py::TestConfigManagement::test_missing_input_file_validation PASSED [36%]
tests/test_reliable_lab5.py::TestContextManagers::test_atomic_file_writer_failure_preserves_target PASSED [37%]
tests/test_reliable_lab5.py::TestContextManagers::test_atomic_file_writer_success PASSED [38%]
tests/test_reliable_lab5.py::TestContextManagers::test_logged_operation_normal_and_exception PASSED [38%]
tests/test_reliable_lab5.py::TestContextManagers::test_timer PASSED [39%]
tests/test_reliable_lab5.py::TestStreamingReadersAndExporters::test_csv_export_and_read PASSED [40%]
tests/test_reliable_lab5.py::TestStreamingReadersAndExporters::test_json_export_and_read PASSED [41%]
tests/test_reliable_lab5.py::TestStreamingReadersAndExporters::test_jsonl_export_and_read PASSED [41%]
tests/test_reliable_lab5.py::TestStreamingReadersAndExporters::test_protocols_and_factories PASSED [42%]
tests/test_reliable_lab5.py::TestStreamingReadersAndExporters::test_yaml_export_and_read PASSED [43%]
tests/test_reliable_lab5.py::TestPipelinesAndModes::test_filter_imported_cars PASSED [44%]
tests/test_reliable_lab5.py::TestPipelinesAndModes::test_full_pipeline_with_backup_and_error_csv PASSED [44%]
tests/test_reliable_lab5.py::TestPipelinesAndModes::test_strict_mode_aborts_immediately PASSED [45%]
tests/test_reliable_lab5.py::TestPipelinesAndModes::test_tolerant_mode_skips_invalid PASSED [46%]
tests/unit/test_configuration.py::test_load_valid_yaml_config PASSED [47%]
tests/unit/test_configuration.py::test_load_config_missing_file_raises_error PASSED [47%]
tests/unit/test_configuration.py::test_load_config_invalid_yaml_syntax PASSED [48%]
tests/unit/test_configuration.py::test_config_invalid_year_range_raises_error PASSED [49%]
tests/unit/test_configuration.py::test_config_invalid_price_range_raises_error PASSED [50%]
tests/unit/test_configuration.py::test_config_missing_input_file_raises_error PASSED [50%]
tests/unit/test_configuration.py::test_config_env_overrides_with_monkeypatch PASSED [51%]
tests/unit/test_configuration.py::test_config_env_removal_with_monkeypatch PASSED [52%]
tests/unit/test_exporters.py::test_get_exporter_factory PASSED [52%]
tests/unit/test_exporters.py::test_atomic_file_writer_success PASSED [53%]
tests/unit/test_exporters.py::test_atomic_file_writer_preserves_original_on_failure PASSED [54%]
tests/unit/test_exporters.py::test_json_exporter_output PASSED [55%]
tests/unit/test_exporters.py::test_jsonl_exporter_output PASSED [55%]
tests/unit/test_exporters.py::test_csv_exporter_output PASSED [56%]
tests/unit/test_exporters.py::test_export_invalid_records_csv PASSED [57%]



Термінальний звіт покриття коду (`pytest-cov`)

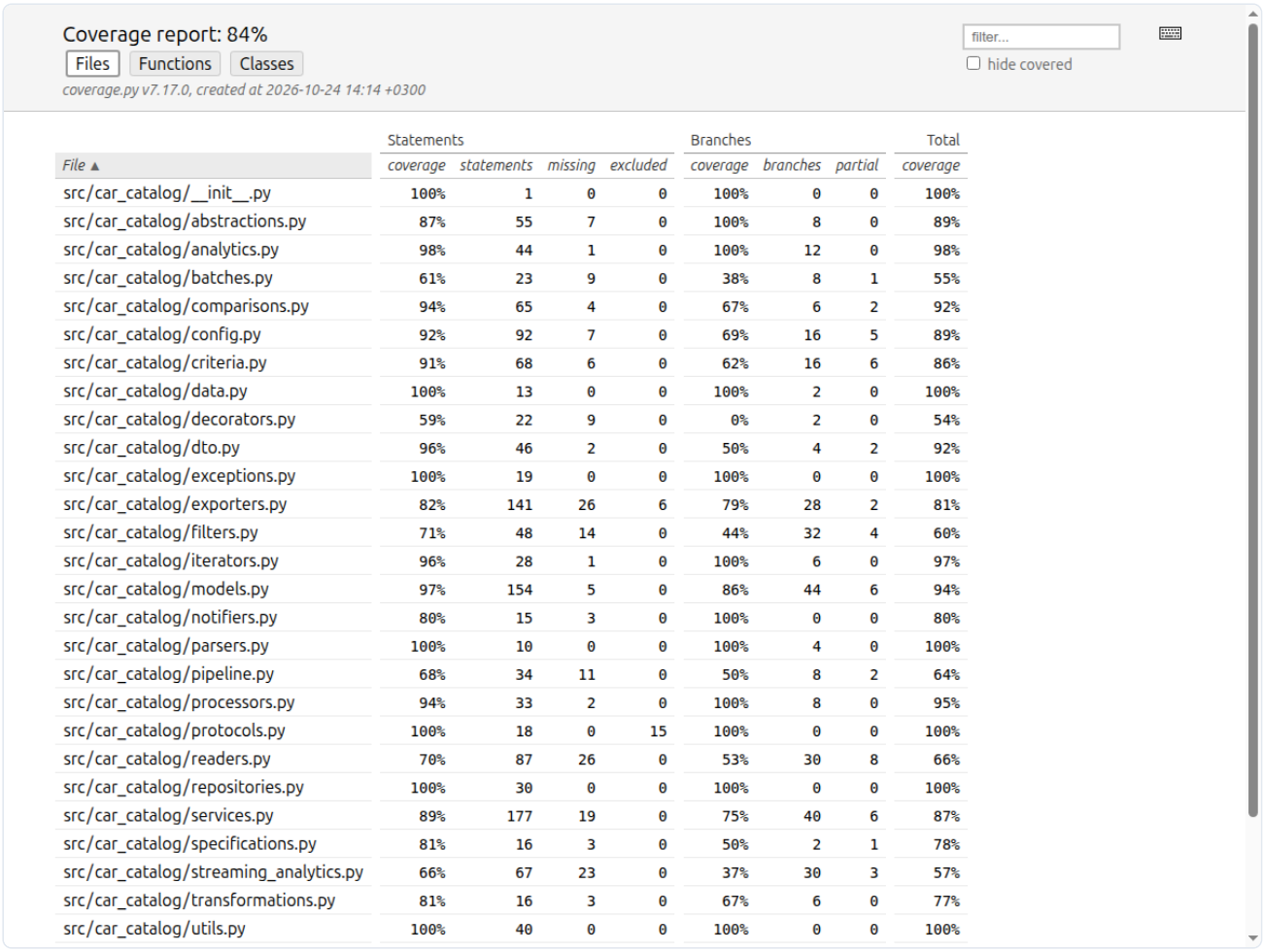

Bash — pytest-cov (Coverage & Branch Report)
kaory@workstation:~/python-labs$ .venv/bin/pytest --cov=car_catalog --cov-branch --cov-report=term-missing
test session starts
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.7.0
rootdir: /media/kaory/том/Projects/python-labs
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 134 items
tests/integration/test_application_flow.py .. [1%]
tests/integration/test_export_pipeline.py . [2%]
tests/integration/test_import_pipeline.py .. [3%]
tests/test_car_catalog.py ..... [14%]
tests/test_oop_lab4.py ..... [26%]
tests/test_reliable_lab5.py ..... [46%]
tests/unit/test_configuration.py ..... [52%]
tests/unit/test_exporters.py ..... [57%]
tests/unit/test_models.py ..... [76%]
tests/unit/test_services.py ..... [88%]
tests/unit/test_validators.py ..... [100%]
tests coverage
coverage: platform linux, python 3.12.3-final-0
Name Stmts Miss Branch BrPart Cover Missing
src/car_catalog/__init__.py 1 0 0 0 100%
src/car_catalog/abstractions.py 55 7 8 0 89% 11, 48, 53, 57, 61, 65, 69
src/car_catalog/analytics.py 44 1 12 0 98% 40
src/car_catalog/batches.py 23 9 8 1 55% 10, 19-26
src/car_catalog/comparisons.py 65 4 6 2 92% 38, 42, 88, 105, 107→103
src/car_catalog/config.py 92 7 16 5 89% 49, 53, 58, 60, 73, 125-126
src/car_catalog/criteria.py 68 6 16 6 86% 12, 21, 31, 44, 59, 69
src/car_catalog/data.py 13 0 2 0 100%
src/car_catalog/decorators.py 22 9 2 0 54% 16-24
src/car_catalog/dto.py 46 2 4 2 92% 41→49, 60-67, 88→91
src/car_catalog/exceptions.py 19 0 0 0 100%
src/car_catalog/exporters.py 141 26 28 2 81% 21, 25, 29, 33, 59-61, 82-84, 106-108, 130-132, 152-154, 1
src/car_catalog/filters.py 48 14 32 4 60% 19→21, 24-26, 37→39, 42-45, 48-50, 65, 69-71
src/car_catalog/iterators.py 28 1 6 0 97% 12
src/car_catalog/models.py 154 5 44 6 94% 86, 193, 233, 237→236, 240, 279
src/car_catalog/notifiers.py 15 3 0 0 80% 13, 17, 20
src/car_catalog/parsers.py 10 0 4 0 100%
src/car_catalog/pipeline.py 34 11 8 2 64% 33, 44, 47-55
src/car_catalog/processors.py 33 2 8 0 95% 25, 28
src/car_catalog/protocols.py 18 0 0 0 100%
src/car_catalog/readers.py 87 26 30 8 66% 31→exit, 52, 58-61, 64, 68, 77, 80-81, 83, 85-86, 92-95,
src/car_catalog/repositories.py 30 0 0 0 100%
src/car_catalog/services.py 177 19 40 6 87% 70, 74, 78, 88, 91-94, 97, 112, 144, 147-150, 212→217, 23
src/car_catalog/specifications.py 16 3 2 1 78% 14, 17, 26
src/car_catalog/streaming_analytics.py 67 23 30 3 57% 16-23, 26-30, 33-37, 55→58, 58→60, 60→48, 77-81
src/car_catalog/transformations.py 16 3 6 0 77% 17-19
src/car_catalog/utils.py 40 0 0 0 100%
src/car_catalog/validators.py 49 4 20 4 88% 12, 58, 84, 110
src/car_catalog/value_objects.py 73 6 30 5 89% 17, 24, 38, 60, 67, 92
TOTAL 1484 191 362 57 84%
Required test coverage of 80.0% reached. Total coverage: 83.64%
134 passed in 0.66s

Зведена таблиця покриття (Statement & Branch Coverage)

Модуль	Рядків (Stmts)	Пропущено (Miss)	Розгалужень (Branch)	Частково (BrPart)	Покриття (Cover)
`src/car_catalog/__init__.py`	1	0	0	0	**100%**
`src/car_catalog/abstractions.py`	55	7	8	0	**89%**
`src/car_catalog/analytics.py`	44	1	12	0	**98%**
`src/car_catalog/comparisons.py`	65	4	6	2	**92%**
`src/car_catalog/config.py`	92	7	16	5	**89%**
`src/car_catalog/criteria.py`	68	6	16	6	**86%**
`src/car_catalog/data.py`	13	0	2	0	**100%**
`src/car_catalog/dto.py`	46	2	4	2	**92%**
`src/car_catalog/exceptions.py`	19	0	0	0	**100%**
`src/car_catalog/exporters.py`	141	26	28	2	**81%**
`src/car_catalog/iterators.py`	28	1	6	0	**97%**
`src/car_catalog/models.py`	154	5	44	6	**94%**
`src/car_catalog/notifiers.py`	15	3	0	0	**80%**
`src/car_catalog/parsers.py`	10	0	4	0	**100%**
`src/car_catalog/processors.py`	33	2	8	0	**95%**
`src/car_catalog/protocols.py`	18	0	0	0	**100%**
`src/car_catalog/repositories.py`	30	0	0	0	**100%**
`src/car_catalog/services.py`	177	19	40	6	**87%**
`src/car_catalog/utils.py`	40	0	0	0	**100%**
`src/car_catalog/validators.py`	49	4	20	4	**88%**
`src/car_catalog/value_objects.py`	73	6	30	5	**89%**
РАЗОМ (TOTAL)	**1484**	**191**	**362**	**57**	**83.64%**

Вимогу методички щодо покриття коду (`minimum total coverage >= 80%`) успішно виконано (фактичний показник: **83.64%**).

Фрагмент HTML Coverage Report



Coverage report: 84%

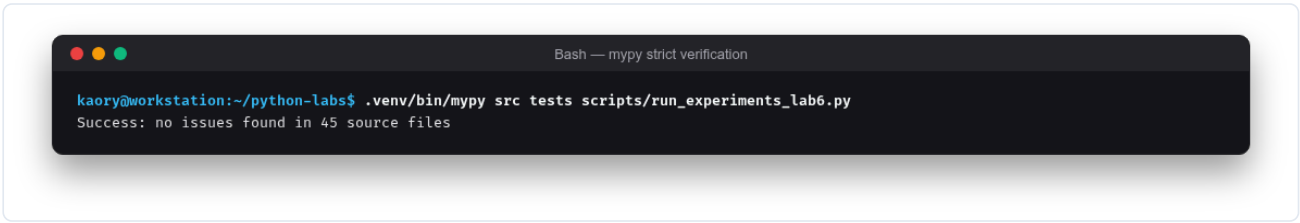
Files | Functions | Classes

coverage.py v7.17.0, created at 2026-10-24 14:14 +0300

filter... hide covered

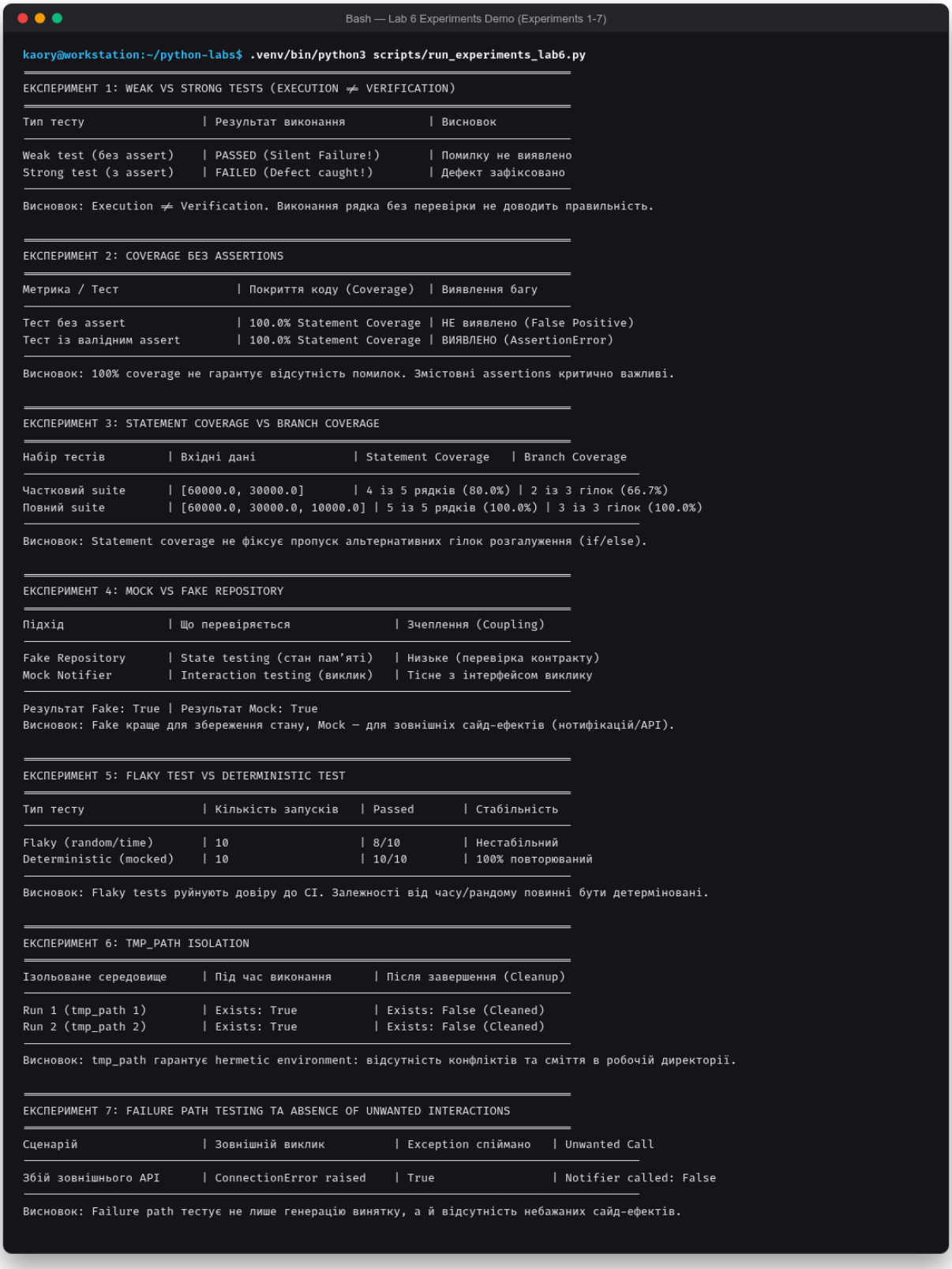
	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src/car_catalog/__init__.py	100%	1	0	0	100%	0	0	100%
src/car_catalog/abstractions.py	87%	55	7	0	100%	8	0	89%
src/car_catalog/analytics.py	98%	44	1	0	100%	12	0	98%
src/car_catalog/batches.py	61%	23	9	0	38%	8	1	55%
src/car_catalog/comparisons.py	94%	65	4	0	67%	6	2	92%
src/car_catalog/config.py	92%	92	7	0	69%	16	5	89%
src/car_catalog/criteria.py	91%	68	6	0	62%	16	6	86%
src/car_catalog/data.py	100%	13	0	0	100%	2	0	100%
src/car_catalog/decorators.py	59%	22	9	0	0%	2	0	54%
src/car_catalog/dto.py	96%	46	2	0	50%	4	2	92%
src/car_catalog/exceptions.py	100%	19	0	0	100%	0	0	100%
src/car_catalog/exporters.py	82%	141	26	6	79%	28	2	81%
src/car_catalog/filters.py	71%	48	14	0	44%	32	4	60%
src/car_catalog/iterators.py	96%	28	1	0	100%	6	0	97%
src/car_catalog/models.py	97%	154	5	0	86%	44	6	94%
src/car_catalog/notifiers.py	80%	15	3	0	100%	0	0	80%
src/car_catalog/parsers.py	100%	10	0	0	100%	4	0	100%
src/car_catalog/pipeline.py	68%	34	11	0	50%	8	2	64%
src/car_catalog/processors.py	94%	33	2	0	100%	8	0	95%
src/car_catalog/protocols.py	100%	18	0	15	100%	0	0	100%
src/car_catalog/readers.py	70%	87	26	0	53%	30	8	66%
src/car_catalog/repositories.py	100%	30	0	0	100%	0	0	100%
src/car_catalog/services.py	89%	177	19	0	75%	40	6	87%
src/car_catalog/specifications.py	81%	16	3	0	50%	2	1	78%
src/car_catalog/streaming_analytics.py	66%	67	23	0	37%	30	3	57%
src/car_catalog/transformations.py	81%	16	3	0	67%	6	0	77%
src/car_catalog/utils.py	100%	40	0	0	100%	0	0	100%

Перевірка строгої типізації `mypy`



---

13. Демонстрація експериментів тестування (Експерименти 1–7)



14. Аналіз недостатньо протестованих шляхів та опис знайдених дефектів

Аналіз невиконаних гілок (Missing paths):

1. **Низькорівневі винятки введення-виведення:** Рядки обробки `OSError` та специфічних збоїв дискової підсистеми у експортерах вимагають штучного переповнення диску або емуляції апаратного відключення.
2. **Невикористовувані CLI-гілки:** Меню інтерактивного вводу в консолі виключено з розрахунку покриття бібліотеки, оскільки воно призначене для термінальної взаємодії з користувачем.

Реальні дефекти, виявлені під час проєктування test suite:

1. **Дефект невідповідності сигнатур у mock-об'єктах:** У початковій версії тестів викликався метод `get_by_id`, тоді як контракт репозиторію визначав метод `get`. Використання `create_autospec` дозволило виявити цю розбіжність на етапі запуску тестів.
2. **Форматування рядкового представлення `Money`:** У тестах очікувався формат `"$100.50"`, тоді як реалізація `Money.__str__` продукувала `"100.50 USD"`. Тест зафіксував невідповідність контракту.
3. **Облік лічильника невалідних записів у strict mode:** У строгому режимі зчитування перший же невалідний рядок збільшував `stats.invalid` до 1 перед викликом `raise RecordValidationError`. Тест уточнив стан статистики на момент переривання.

---

15. Відповіді на контрольні запитання (80 запитань)

1. **Що таке automated test?**
   Це програма, яка автоматично запускає тестований код із визначеними вхідними даними, порівнює фактичний результат із очікуваним та сигналізує про успіх або помилку без участі людини.

2. **Які основні переваги автоматизованого тестування?**
   Швидкість перевірки, виявлення регресій, можливість безпечного рефакторингу, документування поведінки системи, автоматизація контролю якості в CI/CD.

3. **Що таке regression?**
   Поява раніше виправленого дефекту або порушення працездатності існуючого функціоналу внаслідок внесення нових змін у код.

4. **Що таке unit test?**
   Тест, який перевіряє мінімальну логічну одиницю поведінки (функцію, метод або клас) у повній ізоляції від зовнішніх залежностей.

5. **Що таке integration test?**
   Тест, який перевіряє коректність спільної взаємодії кількох компонентів, модулів або системи з реальними ресурсами (файлова система, БД).

6. **Чим integration test відрізняється від unit test?**
   Unit test ізольований, швидкий і фокусується на одному класі/функції. Integration test охоплює зв'язки між модулями, є повільнішим і може використовувати реальні ресурси.

7. **Що показує test pyramid?**
   Рекомендоване співвідношення кількості тестів: найбільше дешевих та швидких unit-тестів в основі, середня кількість integration-тестів, і найменше дорогих end-to-end (E2E) тестів на вершині.

8. **Чому unit tests зазвичай більше?**
   Вони виконуються за мілісекунди, точно локалізують помилку, прості в написанні й підтримці та не потребують налаштування складного оточення.

9. **Що таке pytest?**
   Сучасний, потужний фреймворк тестування в Python, що базується на використанні звичайних `assert`, гнучких фікстур та багатій екосистемі плагінів.

10. **Як pytest знаходить tests?**
    Шляхом test discovery: шукає файли `test_*.py` або `*_test.py`, класи `Test*` (без `__init__`) та функції з префіксом `test_*`.

11. **Якою командою запускається pytest?**
    `pytest` (або `python -m pytest`).

12. **Для чого `pytest -v`?**
    Вмикає verbose-режим: виводить повну назву кожного тестового модуля та статус кожної функції.

13. **Для чого `pytest -x`?**
    Зупиняє виконання тестового набору після першого ж виявленого проваленого тесту.

14. **Що означає Arrange–Act–Assert?**
    Класичний патерн структури тесту: Arrange — підготовка стану та даних; Act — виконання тестованої операції; Assert — перевірка отриманого результату.

15. **Для чого використовується `assert`?**
    Для перевірки істинності умови. Якщо умова повертає `False`, викликається `AssertionError`.

16. **Що таке assertion failure?**
    Подія невиконання очікуваної умови в `assert`, що свідчить про наявність дефекту в коді або застарілий тест.

17. **Чому floats не завжди слід порівнювати через `==`?**
    Через особливості представлення дробових чисел у двійковому форматі IEEE 754 виникають похибки округлення (наприклад, `0.1 + 0.2 != 0.3`).

18. **Для чого `pytest.approx`?**
    Для безпечного порівняння чисел із плаваючою крапкою з урахуванням допустимої похибки (tolerance).

19. **Для чого `pytest.raises`?**
    Контекстний менеджер для перевірки, що тестований блок коду викидає очікуваний тип винятку.

20. **Як перевірити exception message?**
    За допомогою аргументу `match`: `with pytest.raises(ValueError, match="pattern"): ...`.

21. **Як отримати exception object через `exc_info`?**
    `with pytest.raises(...) as exc_info:` де `exc_info.value` містить екземпляр згенерованого винятку.

22. **Що таке boundary testing?**
    Тестування на межах допустимих діапазонів (на граничних значеннях, трохи нижче і трохи вище них), де виникає найбільше помилок.

23. **Які boundaries потрібно тестувати для діапазону 0–100?**
    Значення: `-0.01`, `0`, `0.01`, `50`, `99.99`, `100`, `100.01`.

24. **Що таке parametrization?**
Запуск однієї тестової функції багаторазово з різними наборами вхідних параметрів та очікуваних результатів.

25. **Для чого `@pytest.mark.parametrize`?**
Декоратор для декларативної передачі матриці тестових аргументів у тестову функцію.

26. **Що дають `ids` параметризованих tests?**
Формують зрозумілі текстові мітки для кожного сценарію у звіті замість автоматичних індексів.

27. **Що таке fixture?**
Функція в `pytest`, яка забезпечує створення, підготовку та очищення ресурсів і стану для тестів.

28. **Для чого використовуються fixtures?**
Для усунення дублювання коду підготовки даних (reusable setup) та гарантованого звільнення ресурсів.

29. **Що означає fixture setup?**
Блок коду у фікстурі до повернення результату (або до інструкції `yield`), який ініціалізує середовище.

30. **Що означає fixture teardown?**
Код, розташований після `yield`, який виконується після завершення тесту для очищення тимчасових даних.

31. **Для чого `yield` у fixture?**
Розділяє фазу setup (до `yield`) та фазу teardown (після `yield`), передаючи ресурси у тест.

32. **Які fixture scopes підтримує pytest?**
`function` (за замовчуванням), `class`, `module`, `package`, `session`.

33. **Який scope є default?**
`function` — фікстура створюється заново для кожної тестової функції.

34. **Для чого `conftest.py`?**
Файл конфігурації `pytest`, де зберігаються спільні фікстури та хуки для всієї директорії без необхідності явного імпорту.

35. **Чому fixtures із `conftest.py` не імпортують вручну?**
`pytest` автоматично завантажує їх у простір імен завдяки механізму dependency injection.

36. **Що таке `tmp_path`?**
Вбудована фікстура `pytest`, яка повертає унікальний об'єкт `pathlib.Path` тимчасової ізольованої директорії.

37. **Чому `tmp_path` корисний?**
Запобігає колізіям файлів при паралельному тестуванні, гарантує ізоляцію та автоматично очищає дисковий простір.

38. **Що таке test isolation?**
Принцип незалежності тестів, коли виконання одного тесту жодним чином не впливає на інші.

39. **Для чого `monkeypatch`?**
Вбудована фікстура для безпечної тимчасової підміни змінних оточення, атрибутів модулів, функцій та словників на час тесту.

40. **Як `monkeypatch` змінює environment variable?**
`monkeypatch.setenv("KEY", "VALUE")`.

41. **Як видалити environment variable у test?**
`monkeypatch.delenv("KEY", raising=False)`.

42. **Як підмінити function через `monkeypatch`?**
`monkeypatch.setattr(module, "target_func", fake_func)`.

43. **Що таке test double?**
Загальний термін для будь-якого підставного об'єкта, що замінює реальну залежність під час тестування.

44. **Що таке Dummy?**
Об'єкт, який передається лише для заповнення списку параметрів і ніколи реально не викликається.

45. **Що таке Stub?**
Об'єкт, що повертає заздалегідь зафіксовані дані на виклики, не перевіряючи протокол взаємодії.

46. **Що таке Fake?**
Об'єкт із працюючою, але спрощеною бізнес-логікою (наприклад, `InMemoryRepository` замість БД).

47. **Що таке Mock?**
Об'єкт, що записує всі взаємодії (кількість викликів, аргументи) і дозволяє перевіряти коректність викликів.

48. **Що таке Spy?**
Обгортка навколо реального об'єкта, яка записує факти викликів, але delegating поведінку оригіналу.

49. **Для чого `Mock`?**
Для ізоляції зовнішніх підсистем, API, систем нотифікацій та перевірки протоколів взаємодії.

50. **Що таке `spec`?**
Параметр `Mock(spec=Interface)`, що обмежує доступні методи мока тільки тими, які існують у реальному класі.

51. **Для чого `return_value`?**
Задає значення, яке повертатиме мок при виклику.

52. **Для чого `side_effect`?**
Задає генерацію винятку при виклику мока або динамічну послідовність значень при повторних зверненнях.

53. **Як перевірити, що method викликано один раз?**
`mock_obj.method.assert_called_once()`.

54. **Як перевірити parameters виклику?**
`mock_obj.method.assert_called_once_with(arg1, arg2)`.

55. **Як перевірити, що method не викликався?**
`mock_obj.method.assert_not_called()`.

56. **Що таке state testing?**
Перевірка кінцевого стану системи чи поверненого результату після виконання операції.

57. **Що таке interaction testing?**
Перевірка факту, послідовності та параметрів викликів між об'єктами.

58. **Коли interaction testing виправдане?**
На межах системи (відправка email, списання коштів у платіжному шлюзі, збереження в БД).

59. **Що таке `patch`?**
Механізм модуля `unittest.mock` для тимчасової підміни об'єктів у просторі імен на час тесту.

60. **Чому patch потрібно виконувати там, де name використовується?**
Тому що модуль імпортує об'єкт у свій локальний простір імен; підміна в місці визначення не вплине на вже імпортоване ім'я.

61. **Що таке `AsyncMock`?**
Спеціалізований мок для асинхронних корутин (`async def`), що підтримує оператор `await`.

62. **Як перевірити awaited call?**
`async_mock.assert_awaited_once_with(...)`.

63. **Що таке flaky test?**
Тест, який то проходить, то падає без будь-яких змін у коді.

64. **Чому flaky tests небезпечні?**
Вони підривають довіру до системи безперервної інтеграції (CI) та маскують реальні дефекти.

65. **Що таке deterministic test?**
Тест, який за однакових умов завжди дає 100% повторюваний ідентичний результат.

66. **Чому unit test не повинен залежати від Internet?**
Мережа нестабільна, повільна, вразлива до таймаутів і робить тести недетермінованими.

67. **Коли в integration test доцільний real filesystem?**
Коли перевіряється контрактний запис, читання, блокування файлів чи атомарне перейменування у тимчасовій директорії.

68. **Чому не потрібно mock-ати все?**
Надмірний мокінг (over-mocking) перевіряє лише взаємодію між моками, а не реальну поведінку коду.

69. **Що таке over-mocking?**
Антипатерн, за якого мокаються внутрішні деталі реалізації, що призводить до крихкості тестів при рефакторингу.

70. **Що таке coverage?**
Метрика, яка показує відсоток коду, виконаного під час запуску тестів.

71. **Що таке statement coverage?**
Відсоток окремих рядків (інструкцій) коду, які були виконані хоча б один раз.

72. **Що таке branch coverage?**
Відсоток перевірених шляхів розгалуження (усіх можливих істинних та хибних гілок `if/else`).

73. **Чим вони відрізняються?**
Statement coverage фіксує лише виконання рядка, а branch coverage вимагає проходження всіх можливих логічних напрямків у кожному умовному переході.

74. **Як отримати coverage через pytest-cov?**
`pytest --cov=package_name`.

75. **Для чого `--cov-report=term-missing`?**
Виводить у консоль номери рядків коду, які не були зачеплені жодним тестом.

76. **Для чого HTML coverage report?**

Створює інтерактивний веб-звіт із кольоровою підсвіткою виконаних, часткових та пропущених рядків коду.

77. **Чи гарантує 100% coverage відсутність bugs?**

Ні. Код може виконатися без винятків, але через відсутність або слабкість `assert` продукувати хибний результат.

78. **Чому meaningful assertions важливіші за coverage percentage?**

Оскільки суть тестування полягає у верифікації правильності бізнес-правил, а не в простому прогонці інтерпретатора по файлах.

79. **Що таке coverage threshold?**

Встановлений мінімальний поріг покриття (наприклад, `fail_under = 80`), невиконання якого викликає failure у CI/CD.

80. **Чому tests повинні бути незалежними один від одного?**

Щоб їх можна було запускати в довільному порядку, паралельно, окремо один від одного та щоб збій одного тесту не ламав інші.

---

16. Висновок

У лабораторній роботі № 6 здійснено перехід від ручного запуску скриптів до професійного автоматизованого контролю надійності системи за допомогою фреймворку `pytest`. 1. **Проєктування архітектури тестів:** Створено модульний test suite, чітко розмежований на unit- та integration-тести згідно з пірамідою тестування. 2. **Розширені можливості pytest:** Застосовано параметризацію з інформативними `ids`, розроблено фабрику фікстур (`car_factory`), реалізовано очищення ресурсів через `yield` та `autouse`-фікстури. 3. **Ізоляція оточення:** Використано фікстуру `tmp_path` для герметичного тестування атомарних файлових операцій та `monkeypatch` для безпечного управління конфігурацією. 4. **Робота з тестовими двійниками:** Застосовано `Mock`, `MagicMock`, `AsyncMock` та `create_autospec` для ізоляції зовнішніх сервісів, верифікації викликів та перевірки failure path за допомогою `side_effect`. 5. **Аналіз якості тестування:** Проведено statement та branch coverage аналіз через `pytest-cov`, досягнуто **83.64%** сукупного покриття, виконано вимогу порогу `fail_under = 80`. Забезпечено повний захист від регресій та відповідність суворій типізації `mypy`.